DATA SCIENCE & ANALYTICS · SESSION 03

Data Cleaning & Wrangling

Turning messy, real-world data into something you can actually trust

Asst.Prof.Dr. Chaiwat N. · 2026

2026 · Bachelor Program · 3 hrs · Workshop Format · Hands-on Coding

Session Agenda

01 **Why Data Is Messy**
The shapes dirty data takes — and what it costs you

02 **Missing Values**
isnull, dropna, fillna, and choosing an imputation

03 **Duplicates & Types**
drop_duplicates, astype, to_datetime, string fixes

04 **Reshaping**
merge, concat, melt, and pivot_table

05 **The Cleaning Pipeline**
Wrap every step into one reusable function

06 **Workshop Block**
Clean a messy student table end to end — graded

01

PART 01

Why Data Is Messy

Real data never arrives clean. Before any analysis, you have to find and fix what's broken — and budget for the fact that this is most of the work.

Dirty data doesn't announce itself — it quietly produces a confident wrong answer

Why this session matters

The reality

Surveys put 60–80% of a data scientist's time on cleaning. It isn't a detour before the 'real' work — it is the real work.

Garbage in

A model or chart on dirty data looks just as polished as one on clean data. The error is invisible until someone acts on it.

Today's dataset

A deliberately messy 120-row student table: missing ages, duplicate IDs, prices stored as text, inconsistent faculty spellings.

The goal

Leave today with a repeatable cleaning routine you run on every new dataset — not a one-off set of hacks.

Source: DS Course, Session 3, 2026

Five ways data goes wrong — learn to spot each on sight

The usual suspects

Missing values	Blanks, NaN, None, or sentinel codes like -999. Each needs a deliberate decision, not a silent drop.
Duplicates	The same record entered twice inflates every count and average. Often hides behind a slightly different key.
Wrong types	A price stored as text won't sum; a date as text won't sort. The #1 silent source of broken analysis.
Inconsistent categories	'Eng', 'Engineering', 'ENGINEERING' are three groups to pandas but one to you. They split your groupby.

Source: DS Course, Session 3, 2026

02

PART 02

Missing Values

Find them, understand why they're missing, then decide per column whether to drop the row, fill a sensible value, or leave it flagged.

Step one: find the missing values before you decide anything

See the holes first

Count per column

df.isnull().sum() gives missing-value counts for every column at once — your map of where the holes are.

As a share

Divide by len(df) for the percentage. A column that's 2% missing is very different from one that's 60% missing.

Look before you leap

Never fill or drop blind. Why is it missing? A missing exam score may mean 'absent', not 'unknown'.

PYTHON · FIND MISSING

```
# count missing per column
df.isnull().sum()

# as a percentage of rows
(df.isnull().mean() * 100).round(1)

# rows with ANY missing value
df[df.isnull().any(axis=1)]
```

isnull().sum() is always your first cleaning move.

dropna vs fillna: remove the row, or fill the gap — choose on purpose

Two tools, different costs

dropna

Removes rows (or columns) with missing values. Safe but shrinks your data — dangerous if the gaps aren't random.

fillna

Replaces blanks with a value: a constant, the column mean/median, or the previous value (ffill) for ordered data.

The trade-off

Dropping loses information; filling invents it. State which you chose and why — it changes your results.

PYTHON · DROP vs FILL

```
# drop rows missing critical fields
df.dropna(subset=['gpa'])

# fill numeric gaps with the median
df['age'] = df['age'].fillna(
    df['age'].median())

# carry last value forward (ordered data)
df['price'] = df['price'].ffill()
```

median resists outliers; mean is pulled by them.

Choosing an imputation: the fill you pick encodes an assumption

Match the method to the column

Mean / median

For numeric columns missing at random. Median when the column is skewed or has outliers — it won't be dragged.

Mode

For categoricals — fill with the most common value. Use sparingly; it strengthens whatever's already dominant.

Forward / back fill

For time-ordered data where the last known value is a fair guess — a price, a sensor reading, a status.

Flag, don't fill

Sometimes 'missing' is itself the signal. Add an is_missing column so the model can use it rather than hiding it.

Source: DS Course, Session 3, 2026

03

PART 03

Duplicates & Types

Remove records that were entered twice, and force every column into the type that lets it compute, sort, and group correctly.

Duplicates: find them, understand the key, then drop

One record, counted once

Detect

df.duplicated() flags rows that repeat. Pass a subset of key columns to define what 'the same record' means.

Drop

drop_duplicates() keeps the first occurrence by default. keep='last' or a sort beforehand controls which one survives.

Watch out

A true duplicate and a real repeat purchase look identical. Define the key carefully before you delete anything.

PYTHON · DUPLICATES

```
# how many full-row duplicates?
df.duplicated().sum()

# duplicates by a business key
df.duplicated(subset=['student_id']).sum()

# keep the first, drop the rest
df = df.drop_duplicates(
    subset=['student_id'], keep='first')
```

Define the key before dropping — or lose real data.

Types & standardization: make every column the right dtype

Numbers compute, dates sort

Convert types

astype() coerces a column: text '3.4' to float, an int to category. to_datetime parses date strings into real dates.

Clean strings

Strip whitespace and unify case so 'Eng ' and 'eng' collapse into one value before you group on it.

Verify after

Re-run df.info(). Numbers should be int/float, dates datetime64, categories object — not everything 'object'.

PYTHON · TYPES & STRINGS

```
# text -> number, dates -> datetime
df['gpa']  = df['gpa'].astype(float)
df['date'] = pd.to_datetime(df['date'])

# standardize a category column
df['faculty'] = (df['faculty']
    .str.strip()
    .str.title())   # 'eng ' -> 'Eng'
```

Fix dtypes once, up front — everything downstream relies on it.

04

PART 04

Reshaping

Combine tables and switch between wide and long layouts — merge, concat, melt, and pivot_table move data into the shape your analysis needs.

merge: join two tables on a shared key, like a database JOIN

Bring tables together

The idea

merge lines up rows from two frames by a common column — students with their exam results, orders with customers.

how=

'inner' keeps only matches; 'left' keeps all of the left table. The wrong choice silently drops or duplicates rows.

Check the shape

Compare row counts before and after. A merge that multiplies rows means your key wasn't unique on one side.

PYTHON · MERGE

```
scores = pd.DataFrame({
    'student_id': [1, 2, 3],
    'score': [88, 72, 95]})

df = df.merge(scores,
              on='student_id',
              how='left')

# left join: every student kept
```

Always check df.shape before and after a merge.

concat, melt, and pivot: stack, lengthen, and widen

Reshape, don't recompute

concat	Stacks frames top-to-bottom (more rows) or side-by-side (more columns). Use it to combine batches with the same schema.
melt	Turns wide into long: many value columns collapse into one 'variable' + 'value' pair. Tidy data most charts expect.
pivot_table	The inverse: long back to a wide grid, aggregating on the way. The cross-tab you met in Session 2.
Why it matters	Most 'I can't make this chart' problems are really 'my data is in the wrong shape' problems.

Source: DS Course, Session 3, 2026

05

PART 05

The Cleaning Pipeline

Wrap every step into one function so cleaning is repeatable, reviewable, and applied identically to new data — not a pile of one-off cells.

A cleaning pipeline turns a pile of fixes into one reusable, trustworthy step

Make it repeatable

The problem

Ad-hoc cells run in the wrong order, get skipped, or differ between datasets. Next month you can't reproduce your own result.

The fix

Put every step — types, missing, duplicates, standardization — inside one clean(df) function that returns a clean frame.

The payoff

Run it on any new file with one line. Fix a bug in one place and every dataset gets the fix. Auditable and honest.

Order matters

Types first, then duplicates, then missing values, then derived columns. A wrong order quietly changes the output.

Source: DS Course, Session 3, 2026

One function, the whole routine — your reusable cleaning pipeline

Cleaning as code

Define once

Each line is a step you already know. Reading the function top to bottom is a readable spec of your cleaning.

Return clean

The function takes a raw frame and returns a clean copy — the original is never mutated, so you can re-run safely.

Reuse forever

clean(raw) on this dataset; clean(new_file) next month. Same logic, zero copy-paste.

PYTHON · PIPELINE

```
def clean(df):
    df = df.copy()
    df['gpa'] = df['gpa'].astype(float)
    df = df.drop_duplicates('student_id')
    df['age'] = df['age'].fillna(
        df['age'].median())
    df['faculty'] = df['faculty'].str.title()
    return df

clean_df = clean(raw)
```

Wrap the steps once; run them everywhere.

06

PART 06

Workshop Block

Take a deliberately messy student table and return a clean one — duplicates gone, types fixed, gaps handled — as a single clean() function. Graded.

By the end of this block, every student has a clean() function that fixes the messy table

4 Steps · 45 Minutes

Audit	Run isnull().sum(), duplicated().sum(), and info(). Write down what's broken before touching anything.
Fix types	Convert gpa to float and any date column to datetime. Strip and title-case the faculty strings.
Handle gaps & dupes	Drop duplicate student_ids; fill missing ages with the median. Justify each choice in a comment.
Wrap it up	Put every step inside one clean(df) function and prove it: clean(raw).isnull().sum() is all zeros.

Source: Workshop, Session 3, 2026

Your deliverable: a messy frame in, a clean frame out, one function

What 'done' looks like

The task

A single clean() function that an auditor could read top to bottom and trust. Clear comments on every judgment call.

Graded on

Did it run? Is the output actually clean — no dupes, right dtypes, no unexpected NaN? Is each choice justified?

Finish early?

Add a print of the before/after shape, and a flag column marking which ages you imputed.

PYTHON · DELIVERABLE

```
clean_df = clean(raw)

print('Before:', raw.shape)
print('After :', clean_df.shape)
print('Missing:', clean_df.isnull().sum().sum())
print('Dupes  :', clean_df
       .duplicated('student_id').sum())
# Missing: 0   Dupes: 0
```

Messy in, clean out, fully reproducible.

KEY TAKEAWAY

Clean data is most of the job. Do it on purpose, not by accident.